

```
x=1  
y=x+1  
x=2
```

In Reactive Programming,  $y$  deve continuare ad avere il valore aggiornato seguendo una regola, cioè  $y$  deve essere sempre  $x+1$  (quindi, in questo blocco,  $y=3$ ). Quindi,  $y$  è una variabile reattiva perchè, in qualche modo, reagisce al cambiamento del valore di  $x$  aggiornandosi automaticamente perchè viene definita una relazione (assegnamento definisce una relazione tra quantità ed ogni volta che cambia l'espressione che ho sulla destra, con cui ho definito una certa variabile, viene aggiornata automaticamente la variabile di sinistra). Questo approccio ha un livello di astrazione che mi permette di evitare di parlare di eventi. Nell'esempio, viene dichiarato che da adesso in avanti tutte le volte che cambia  $x$  automaticamente si propaga il cambiamento sulla  $y$ , essendo  $y=x+1$ . E' utile vedere le variabili come dei flussi di valori che cambiano nel tempo (in modo asincrono) e con il fatto che parte di questi flussi si aggiornano automaticamente essendo stati definiti in relazione ad altri flussi (come composizione di altri flussi)

# Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO

a.a 2025/2026

Prof. Alessandro Ricci

[module-2.2]

REACTIVE PROGRAMMING

# SUMMARY

- Reactive programming (RP) overview
  - research roots
    - reference paper:
      - Engineer Bainomugisha, Andoni Lombide Carreton, Tom van Cutsem, Stijn Mostinckx, and Wolfgang de Meuter. 2013. *A survey on reactive programming*. ACM Comput. Surv. 45, 4, Article 52 (August 2013)
    - basic abstractions
      - behaviours and events
- RP in the mainstream: Reactive extensions (Rx)
  - Observable, observers

# REACTIVE PROGRAMMING

Mentre approcci come i Callback (CPS) o le Promises sono progettati per gestire una singola operazione asincrona che restituisce un risultato una sola volta, la programmazione reattiva è fatta per gestire sequenze di eventi nel tempo.

- Both the CPS and Promises <sup>partono dal presupposto che</sup> ~~assume that~~ asynchronous computations will deliver their result in *one shot*
  - however <sup>comune</sup> it is ~~not uncommon~~ in application the need to manage (asynchronous) **streams** of data/events.
  - this is the goal of **Reactive Programming**
    - asynchronous data stream programming <sup>perché immaginate che le variabili possono essere degli stream asincroni di dati, cioè flussi di dati asincroni che cambiano nel tempo e che possiamo comporre</sup>
- Reactive Programming paradigm
  - <sup>focalizzato su</sup> ~~oriented around~~ **data flows** and the propagation of change
  - <sup>semplifica la gestione</sup> ~~easing the management~~ of asynchronous streams of data and events
    - strongly related to the Observer pattern & event-driven programming
  - originally introduced in 1990s, it is attracting much attention today as an effective approach for developing responsive (web) apps, in particular in the context of Big data

La differenza principale sta nella quantità e nella durata:

- Promises/CPS: Utili per operazioni "one-shot" (es. una singola richiesta HTTP). Una volta ottenuto il risultato, l'operazione è conclusa.

- Reactive Programming: Utile per flussi continui. Esempi classici sono i movimenti del mouse, i messaggi in una chat websocket, o i dati provenienti da un sensore IoT. Il sistema rimane "in ascolto" e reagisce ogni volta che arriva un nuovo dato.

# RP LANGUAGES AND FRAMEWORKS

I linguaggi hanno la RP all'interno del linguaggio (es: variabili reattive), altri hanno dei framework

- *Reactive extensions* have been introduced for all major programming languages/systems
  - FrTime (based on Scheme)
  - Microsoft's Reactive Extensions (Rx) RxJava, RxScala, etc.. (organizzare un programma con queste astrazioni anche se sotto il linguaggio non è reattivo)
  - Reactive JavaScript - Flapjax, Bacon.js, AngularJs
  - Reactive DART
  - Scala.React
  - RxJava
  - ...

Alcuni componenti, in generale, li dichiaro che sono legati, in modo reattivo, al cambiamento di altri.

La programmazione reattiva si basa su alcuni concetti chiave:

- Propagazione del cambiamento: Se ho una variabile  $C$  che dipende da  $A$  e  $B$  ( $C = A + B$ ), in un sistema reattivo, ogni volta che  $A$  cambia, il valore di  $C$  viene aggiornato automaticamente senza dover richiamare la funzione. È lo stesso principio del funzionamento dei fogli di calcolo (Excel).
- Pattern Observer: È l'anima del sistema. C'è un "Observable" (chi emette i dati) e uno o più "Observers" (chi si iscrive per riceverli e reagire).
- Operatori: La vera potenza sta nella possibilità di manipolare i flussi usando operatori (simili a quelli funzionali come map, filter, reduce). Puoi unire due flussi, filtrarne uno, o ritardare l'emissione di dati con estrema facilità.

# REACTIVE PROGRAMMING ROOTS

- Research roots
  - most of the research on reactive programming descends from the language Fran [Elliott and Hudak 1997]
    - a functional domain specific language developed in the late 1990s to ease the construction of graphics and interactive media applications
    - **Functional Reactive Programming (FRP)**
  - based on the *synchronous dataflow programming* paradigm [Lee and Messerschmitt 1987] but with relaxed real-time constraints.
- Recently proposed as a solution to deal with the problems and complexity related to the development of event-driven applications

# ASYNCHRONOUS PROGRAMMING PROBLEMS

Nella programmazione sequenziale (imperativa), il programmatore è il "regista": Fai questo. Poi fai quest'altro. Se succede X, fai Y. Nell'asincronia, il regista è l'ambiente esterno. Il tuo codice non ha più il controllo del flusso (il main thread), ma viene interrotto da eventi che arrivano quando vogliono. Il problema: Il flusso logico diventa frammentato. Invece di leggere il codice dall'alto verso il basso, devi saltare tra decine di event handler (funzioni di callback) sparsi. La conseguenza: È difficilissimo ricostruire mentalmente "cosa sta succedendo" in un dato momento perché la logica di business è spezzettata in tanti piccoli pezzi non collegati tra loro.

- Asynchronous programs and applications
  - difficult to program using conventional sequential programming approaches, because *it is impossible to predict or control the order of arrival of external events*
    - control jumps around event handlers as the outside environment changes unexpectedly
- The ***Inversion of control*** problem
  - the control flow of a program is driven by external events and not by an order specified by the programmer
  - when there is a state change in one computation or data, the programmer *is required to manually update all the others that depend on it*
  - such manual management of state changes and data dependencies is complex and error-prone (e.g., performing state changes at a wrong time or in a wrong order) [Cooper and Krishnamurthi 2006]

# ASYNCHRONOUS PROGRAMMING PROBLEMS

Quando un programma dipende da molti eventi esterni (click dell'utente, risposte di rete, timer, sensori), l'esecuzione non segue più una linea retta dall'alto verso il basso (come nella programmazione sequenziale). Il controllo viene "ceduto" all'ambiente esterno (Inversion of Control). Gestire il modo in cui queste funzioni di risposta (callback) si coordinano tra loro diventa rapidamente difficilissimo.

- Coordinating callbacks can be a ~~very daunting task~~ <sup>compito molto arduo</sup> even for advanced programmers
  - numerous isolated code fragments can be manipulating the same data and their order of execution is unpredictable Poiché gli eventi esterni arrivano quando vogliono, non puoi sapere in che ordine verranno eseguite le callback.
  - “asynchronous spaghetti”
- **Since callbacks usually do not have a return value, they must perform *side effects* in order to affect the application state**

[Cooper 2008]. Programma ad eventi si basano sull'Inversione del Controllo: io faccio qualcosa in relazione ad un evento che viene generato dall'esterno, quindi concepisco la logica del mio componente in termini di reazione a quello che viene dall'esterno (ma queste reazioni devono avere side-effect), C'è un modo per avere stessa reattività ma in modo totalmente dichiarativo senza side-effect (FRP)?

La Programmazione Reattiva trasforma il codice "procedurale" in codice dichiarativo: Invece di dire come aggiornare le dipendenze, dici quali sono le dipendenze. In questo modo, non devi più preoccuparti di "chiamare" le funzioni di aggiornamento manualmente. È il framework Reattivo che, quando il dato arriva (spinto dall'esterno), fa fluire automaticamente la modifica lungo tutta la catena di dipendenze.

# REACTIVE PROGRAMMING PARADIGM

Voglio avere un modello di dichiarazione il più dichiarativo possibile che sia reattivo ai cambiamenti (gestiti automaticamente)

## RP paradigm

Prima dovevi gestire manualmente ogni callback, timer e sincronizzazione asincrona. La RP crea un'astrazione di tempo: "Tu definisci come i dati sono collegati tra loro, alla gestione dei ritardi e della sincronizzazione ci pensa il motore reattivo".

- ~~providing abstractions~~ <sup>fornire astrazioni</sup> to express programs as *reactions* to external events...
- ... but ~~having the language automatically manage the flow of time~~ <sup>lasciando che sia il linguaggio a gestire automaticamente</sup> (by ~~conceptually supporting simultaneity~~ <sup>ai calcoli</sup>), and ~~data and computation dependencies~~ <sup>le dipendenze relative ai</sup>.

I cambiamenti ci sono, ma vengono gestiti "sotto" (ad esempio, in java non ci sono i puntatori, ma sotto il Motore di Java li usa automaticamente senza che il programmatore li usi esplicitamente).

<sup>astrarre la gestione del tempo</sup>

- ~~abstracting over time management~~, just as garbage collectors abstract over memory management

Il motore reattivo astrae il tempo. Tu non dici al sistema quando o come sincronizzare le cose momento per momento; dichiarai solo le relazioni tra i dati, e la RP gestisce l'asse del tempo in sottofondo.

- RP facilitates the ~~declarative development~~ <sup>sviluppo dichiarativo</sup> of event-driven applications

- developers can express programs in terms of what to do, and let the language automatically manage when to do it
- ~~built around the notion of~~ <sup>si basa sul concetto di</sup> ~~continuous time-varying values~~ <sup>valori che variano continuamente nel tempo</sup> and ~~propagation of change~~ <sup>propagazione del cambiamento</sup>

- state changes are automatically and efficiently propagated, across the network of dependent computations, by the underlying execution model.

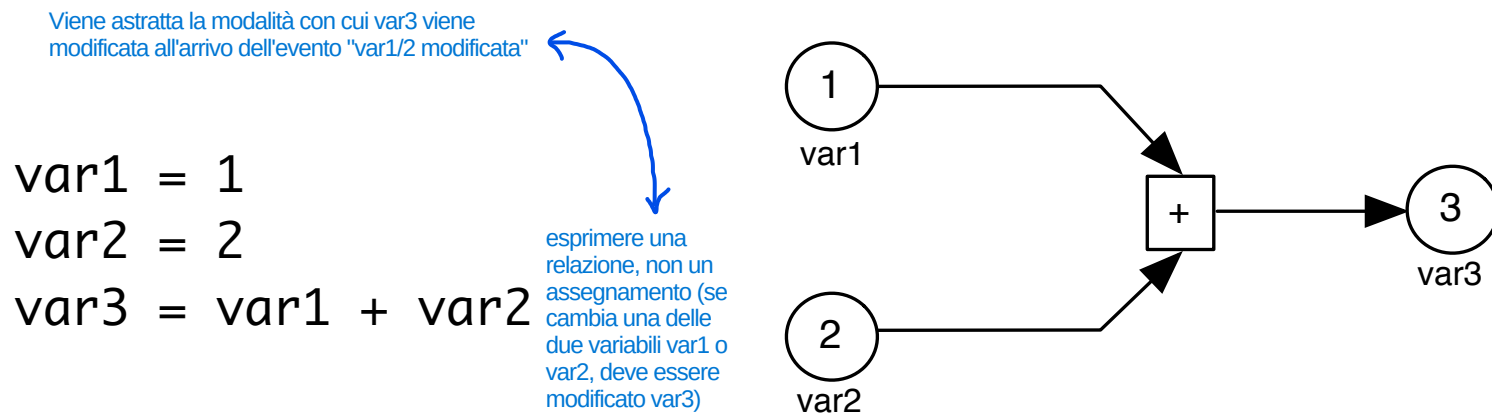
Il dev si occupa solo di definire la mappa delle dipendenze. Sarà il motore di esecuzione (il runtime reattivo) a farsi carico di spingere le modifiche lungo la mappa, aggiornando solo ciò che serve, al momento giusto e nell'ordine perfetto.

ti permette di definire relazioni tra dati con la certezza matematico-logica che tutti i rami di dipendenza si aggiorneranno "all'istante e insieme", senza mostrare mai stati inconsistenti o dati "vecchi e nuovi" mescolati tra loro.



# CHANGE PROPAGATION EXAMPLE

- Consider a simple example of calculating the sum of two variables.



- In reactive programming, the value of the variable var3 is always kept up-to-date.
  - the value of var3 is automatically recomputed over time whenever the value of var1 or var2 changes
- Values change ~~over time~~ <sup>nel tempo</sup> and ~~when they change~~ <sup>quando cambiano</sup>, all dependent computations are automatically re-executed
- In reactive programming terminology, the variable var3 is said to be dependent on the variables var1 and var2

# RP ABSTRACTIONS

Il problema senza astrazione: Tradizionalmente, per gestire gli eventi che cambiano nel tempo, devi scrivere codice complicato: impostare timer con `setInterval`, registrare dei listener per i click, gestire variabili temporanee per ricordarti lo stato precedente, o sincronizzare thread. Il tempo è un fattore "esterno" e caotico che devi gestire manualmente. La soluzione con l'astrazione: La Programmazione Reattiva prende tutti questi eventi che accadono nel tempo e li "confeziona" all'interno di un'astrazione pulita: il Flusso (Stream). Non ti interessa più quando la scheda di rete riceverà un pacchetto o quando l'utente muoverà il mouse: il motore reattivo ti consegna un "tubo" pronto all'uso da cui escono dati nel tempo. Il tempo viene "astratto": Tu non gestisci l'asse temporale, vedi solo la sequenza di dati che arriva dal flusso.

- Reactive programming abstracts ~~time varying events~~ <sup>gli eventi che variano nel tempo per</sup> ~~for their~~ <sup>permetterne l'uso da parte del programmatore</sup> ~~consumption by a programmer~~
  - the programmer can then define elements that will react upon each of these incoming time-varying events <sup>any entity that can be treated exactly like an ordinary data value</sup>
  - abstractions are first-class values, and can be passed around or even composed within the program.

## Two main kinds of reactive abstractions:

è un paradigma che porta a rivedere il modo di modellare i sistemi che abbiamo sotto sempre e comunque in termini di flussi asincroni che possono essere combinati con degli operatori

### event streams

Natura: Discreta e intermittente. Gli eventi accadono in momenti precisi, ma non sai quando. Cosa rappresentano: Azioni isolate. Si usano per gestire input utente, arrivo di messaggi in chat, o notifiche push.

La natura del dato è discreta. Tra un evento e l'altro non c'è un valore "corrente": semplicemente non sta succedendo nulla.

**modeling discrete (or sparse) time-varying values** (vengono modellati degli eventi discreti: ad esempio, è passato un oggetto)

- asynchronous abstractions for the progressive flow of intermittent, sequential data <sup>I dati arrivano uno dopo l'altro nel tempo</sup> coming from a recurring event

- e.g. mouse <sup>click</sup> events can be represented as event streams.

### behaviours (or signals)

Natura: Continua e fluida. In ogni istante il "Behaviour" ha un valore. Non c'è mai un momento di "vuoto". Cosa rappresentano: Uno stato che cambia nel tempo. Si usano per rappresentare lo stato della UI (es. "il colore del bottone dipende dal tempo trascorso") o sensori di pressione/temperatura.

valori continui nel tempo

- continuous values over time <sup>Ha sempre uno stato corrente leggibile in qualsiasi istante t. Non c'è mai un momento di "vuoto" o assenza di valore.</sup>

- abstractions that represent uninterrupted, fluid flow of incoming data from a <sup>evento costante</sup> steady event.

La sorgente ("steady") è sempre attiva e genera o mantiene costante il flusso informativo.

- e.g. a timer can be represented as a behaviour.

Il flusso è come un tubo vuoto in cui, di tanto in tanto, passa un "pacchetto" di dati.

La sorgente è qualcosa che può accadere più volte nel tempo.

Rappresenta il concetto di segnale continuo. Se interroghi un Behaviour in un qualunque microsecondo, esso ti restituirà sempre un valore valido.

# EXAMPLE IN FLAPJAX

Linguaggio di programmazione costruito su JavaScript che poneva delle astrazioni reattive

- A timer in Flapjax

```
var timer = timerB(100);  
var seconds = liftB(  
  function (time){  
    return Math.floor(time / 1000);  
  }, timer );  
insertDomB(seconds, 'timer-div');
```

timerB è come se fosse una Factory: crea un flusso asincrono (Behaviour)

un altro flusso asincrono di tipo Behaviour che dipende dai valori di timer (viene definita la regola di trasformazione da millisecondi a secondi: da un flusso si trasforma il valore per un altro flusso che è seconds)

- This examples set up a DOM element in web page showing a timer count in seconds.

- first a timer behaviour producing a value after every 100 milliseconds is created (timer) il timer è un behaviour: valore continuo nel tempo che cambia incrementalmente

- timer is a behaviour variable

- to show the time in second, a further behaviour variable is defined (seconds), storing the flow in seconds

- finally the "seconds" behaviour is inserted within the DOM element with the ID timer-div on a webpage elemento del DOM associato a "seconds": "timer-div" cambia automaticamente

- the value of the timer will therefore be continuously updated and displayed on the page

- Key point:

- **fully declarative way of managing events and event streams** non vengono gestiti eventi, callback (viene tutto astratto)

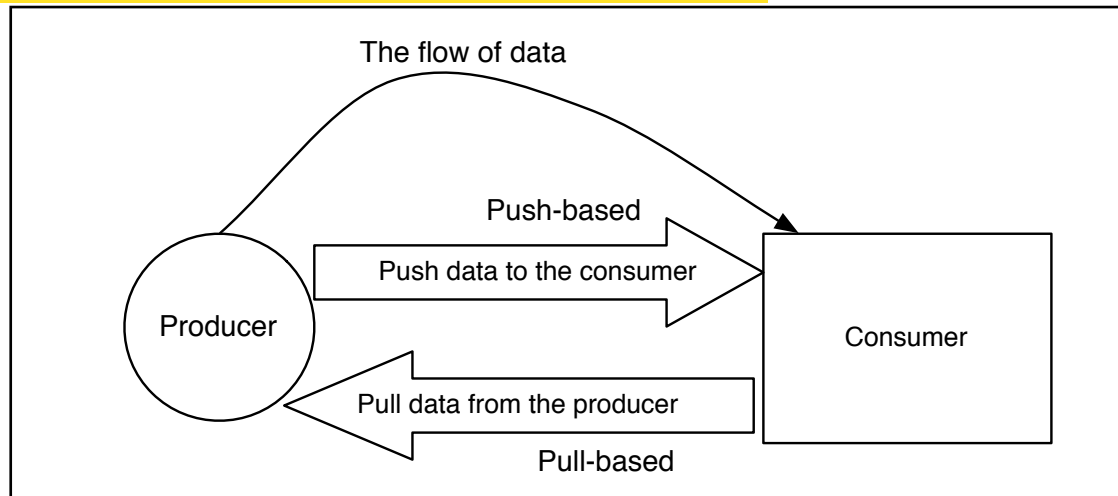
# EVALUATION MODEL

Un Evaluation Model (Modello di Valutazione) è il "motore di esecuzione" di un linguaggio o framework reattivo. Il suo compito principale è definire le regole e i meccanismi con cui i cambiamenti nei dati vengono propagati attraverso il grafo delle dipendenze per ricalcolare i valori necessari e mantenere coerente l'intero sistema.

- The evaluation model of a reactive programming language ~~is concerned with~~<sup>si occupa di:</sup> *how changes are propagated across a dependency graph of values and computations.* modello di valutazione che deve propagare gli eventi: quindi, espresso delle relazioni tra i vari flussi e variabili reattive, dobbiamo aggiornare le variabili in funzione degli eventi.
  - From the programmer's point of view, propagation of changes happens automatically
    - a change of a value should be automatically propagated to all dependent computations
    - when there is an event occurrence at an event source, dependent computations need to be notified about the changes, possibly triggering a recomputation (Announcer-Listener Architecture) (Pattern Observer)
- Quando un ingegnere progetta un linguaggio reattivo o una libreria reattiva, deve decidere come implementare il motore interno (Evaluation Model)
- At the language level, the design decision that needs to be taken into account is **who initiates the propagation of changes**
    - 1 – ~~whether~~<sup>se</sup> the source should “push” new data to its dependents (consumers) → è la computazione dipendente
    - 2 – or the dependents should “pull” data from the event source (producer) → è la sorgente di eventi/dati

# PULL AND PUSH MODELS

le computazioni dipendenti (dependent computations) sono tutti quei nodi, variabili, funzioni o trasformazioni il cui valore finale dipende direttamente o indirettamente dai dati prodotti da una sorgente. In parole semplici: sono i "consumatori" o "anelli intermedi" della catena che devono essere ricalcolati o aggiornati ogni volta che i dati di origine cambiano.



Due modelli per implementare il modello di valutazione reattivo (cioè come avviene la propagazione dei cambiamenti verso il grafo delle dipendenze nel motore del linguaggio reattivo?)

## Two evaluation models

- 1 – **Pull-based** - the computation that requires a value needs to “pull” it from the source.
  - propagation is driven by the demand of new data (demand-driven)
  - can be implemented by lazy languages (e.g. Haskell) modalità Cold in Rx: vado a prendere i dati solo quando mi servono
- 2 – **Push-based** - when the source has new data, it pushes the data to its dependent computations.
  - propagation is driven by availability of new data (data-driven) rather than the demand.
  - implemented by eager languages (e.g. Flapjax based on JavaScript, Scala.React)

# GLITCHES

- Nel modello Push: La sorgente spinge ciecamente i nuovi dati verso i nodi dipendenti appena arrivano. Se la propagazione segue un percorso errato del grafo, alcuni nodi ricevono la "spinta" prima di altri, provocando il glitch.
- Nel modello Pull: Il consumatore finale richiede il dato. Quando var3 viene richiesto, risale l'albero e chiede prima i valori aggiornati a var1 e var2, valutando tutto in modo lazy e garantendo che tutti i valori siano già aggiornati al momento del calcolo.

- Glitches are update inconsistencies that may occur during the propagation of changes when a computation is run before all its dependent expressions are evaluated, it may result in fresh values being combined with stale values, leading to a glitch [Cooper and Krishnamurthi 2006].
- problem affecting only the push-based evaluation model

- Example Se var1 viene aggiornato, tutte le dipendenze devono essere aggiornate in modo consistente. Dobbiamo avere un modello di aggiornamento che garantisce l'atomicità dell'aggiornamento di variabili che dipendono da espressioni che possono avere variabili condivise (innestate)

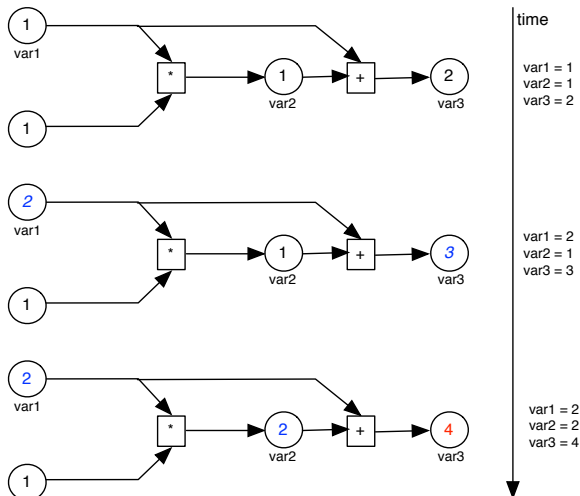
var1 = 1  
var2 = var1 \* 1  
var3 = var1 + var2

Dipendenze innestate

The value of the variable var2 is expected to always be the same as that of var1, and that of var3 to always be twice that of var1. When the value of var1 is 1, the value of var2 is 1 and var3 is 2.

If the value of var1 changes to, say 2, the value of var2 is expected to change to 2 while the value of var3 is expected to be 4. However, in a naive reactive implementation, changing the value of var1 to 2 may cause the expression var1 + var2 to be recomputed before the expression var1 \* 1. Thus the value of var3 will momentarily be 3, which is incorrect. Eventually, the expression var1 \* 1 will be recomputed to give a new value to var2 and therefore the value of var3 will be recomputed again to reflect the correct value 4.

Si costruiscono dei grafi delle dipendenze per avere la garanzia che quando viene modificato un flusso/variabile, tutti gli altri vengono aggiornati consistentemente considerando tutte le dipendenze del caso (quindi, non posso far vedere degli stati del sistema che non siano consistenti con la dichiarazione che abbiamo fatto sulle variabili reattive)



# GLITCH AVOIDANCE

- Most recent reactive implementations achieve glitch avoidance in reactive programs running on a single computer, but not in distributed reactive programs
- Avoiding glitches in a distributing setting is not <sup>easy</sup> straightforward because of network failures, delays and lack of a global clock
  - open research issue [Salvaneschi et al 2014, Mogk et al. 2018]

→ Descrivere delle dipendenze tra flussi di dati che vivono su nodi diversi del sistema distribuito è complicato (Distributed Reactive Programming)



# LIFTING

Nei framework di linguaggi non reattivi, si costruisce, sopra al linguaggio, un layer per cui trasformiamo da variabili non reattive a reattive (processo di conversione chiamato Lifting)

→ se un'espressione usa o dipende da una variabile reattiva (un Behaviour o un Event Stream), anche il risultato di quell'espressione DEVE diventare reattivo.

- The values in expressions that depend on the reactive values *need to be reactive themselves*.
  - A variable which gets assigned with some expression involving behaviours or event streams becomes a reactive variable itself
    - since updates from the behaviours/event streams will affect the variable itself
  - The process of converting a normal variable into a reactive variable is called **lifting**
- Il Lifting è il processo di conversione o elevazione
- Quando "lifti" un'espressione, la libreria reattiva non esegue solo il calcolo, ma aggiunge un nuovo nodo al Grafo delle Dipendenze
- In order to correctly recompute all the reactive expressions once an event stream or behaviour is triggered, most libraries construct a *dependency graph* behind the scenes.
    - whenever an expression changes, the dependent expressions are recalculated and their values updated
    - In the timer example, a time change that happens updating timer triggers the re-evaluation of the function  $(time)\{...\}$  and the update of seconds, which subsequently updates the value inserted in the web page

da una variabile reattiva (Behaviour)

timer è un flusso reattivo che emette valori di tempo ogni 100ms. La funzione anonima  $function(time)\{ return Math.floor(time / 1000); \}$  è una funzione normale, statica. Tramite l'operatore liftB, la funzione viene promossa (liftata): essa viene collegata a timer. Di conseguenza, anche seconds diventa una variabile reattiva. Ogni volta che timer aggiorna il suo valore, il motore reattivo riesegue automaticamente la funzione e propaga il nuovo valore dei secondi verso l'interfaccia grafica (DOM).



# IMPLICIT VS. EXPLICIT LIFTING

- Implicit vs. explicit lifting
  - a number of JavaScript libraries implicitly perform lifting for the programmer – e.g. in Bacon.js.
  - for some, the programmer has to perform lifting of reactive expressions manually – such as in React.js
  - a third category of libraries offer both implicit and explicit lifting – for instance Flapjax
  - which if used as a library the programmer perform lifting explicitly but if used as a compiler the code is transformed implicitly

# COMPOSING EVENT STREAMS AND BEHAVIOURS

Rx: pattern observer asincrono.

In Rx, posso creare dei flussi a partire da altri flussi e questo avviene attraverso degli operatori (ad esempio, la Map): diventano i "mattoncini elementari" per comporre i flussi come vogliamo

Il senso primo della Programmazione Reattiva pura è riuscire a definire dei flussi come composizione di altri flussi che catturano una certa logica di dominio (la logica con cui se non avessi quel livello dovrei fare con delle callback)

- A essential feature in reactive programming is **reactive abstraction composition**
  - this allows to avoid the callback nightmare described previously
  - <sup>example</sup> for instance, instead of having three separate callbacks to separately handle mouse clicks, mouse moves or mouse releases, we can compose them as a single event stream which responds to all the three events.
- Most JavaScript libraries provide this kind of fundamental support.
  - for example, in Bacon.js, properties (or behaviours) can be composed using the combine operator
  - in Flapjax, mergeE function is provided, which throttles the latest stream that comes in from either event.

# COMPOSING EVENT STREAMS

- Example in Flapjax:

extractEventE: operazione di Lifting (associare alla generazione di flussi di eventi ai componenti con cui lo user può interagire nella pagine web)

```
var saveTimer = timerE(10000); //10 seconds
var saveClicked = extractEventE('save-button', 'click');
var save = mergeE(saveTimer, saveClicked);
save.mapE(doSave); //save received data
```

flusso asincrono associato al button click  
come flusso asincrono viene messo, ogni volta che l'utente clicca save, evento click

- In this example three event streams are created:

- saveTimer is an event stream is created, that is triggered after every 10 seconds
- saveClicked is an event stream that is triggered whenever a user clicks on the save-button
- save is triggered whenever either the timer elapses or the save button is clicked
- finally, for each element of the save stream, the doSave function is called (functional style)

un flusso conterrà un evento ogni volta che o passano 10 secondi o viene cliccato il bottone save

(flusso legato allo scorrere del tempo)

un unico flusso asincrono save che è la fusione dei due flussi asincroni saveTimer e saveClick

ogni volta che

scade

descritto dichiarativamente il fatto che voglio salvare ogni 10 secondi o ogni volta che viene cliccato il bottone (senza avere callback sul tempo che passa e sulla persona che clicca sul bottone: lo si fa catturando i flussi asincroni)

mapE: fai in modo che venga chiamata doSave (callback) ogni volta che c'è un evento nel flusso merge. Nell'esempio vengono salvati i dati ad ogni evento ricevuto dal flusso composito

# ONGOING RESEARCH DIRECTIONS

- Among the advanced aspects currently explored in the research:
  - distributed reactive programming
    - composing reactive distributed streams
  - handling fault tolerance

# AVAILABLE REACTIVE LIBRARIES

- Integrating reactive programming in existing programming languages/paradigms
- Main examples
  - **Reactive Extensions (Rx)** la più diffusa
    - an API for asynchronous programming with observable streams, available on multiple platforms
    - <http://reactivex.io/>
  - **Reactive Streams**
    - Initiative to provide a standard for asynchronous stream processing with non-blocking back pressure - for JVM and JavaScript runtime environments
    - <https://www.reactive-streams.org/>
  - **Project Reactor**
    - fourth-generation reactive library, based on the Reactive Streams specification, for building non-blocking applications on the JVM
    - <https://projectreactor.io/>

# COMMON FEATURES AND CONCEPTS [\*]

Componibilità

(La Componibilità è una qualità del linguaggio. La Composizione è l'atto pratico o la tecnica con cui prendi due o più elementi e li unisci per crearne uno nuovo.)

- 1 • **Composability** and **readability**
- 2 • **Data as a flow** manipulated with a rich vocabulary of **operators**
- 3 • **Nothing happens** until you subscribe
- 4 • **Backpressure** or the ability for the consumer to signal the producer that the rate of emission is too high

[\*] From: <https://projectreactor.io/docs/core/release/reference/#getting-started>

1

# COMPOSABILITY & READABILITY

La Reactive Abstraction  
Composition è il modo specifico in cui il paradigma reattivo applica la componibilità agli eventi nel tempo.

- *Composability* <sup>Componibilità</sup> as the ability to <sup>coordinare più</sup> orchestrate multiple asynchronous tasks, <sup>in cui i risultati dei task precedenti</sup> in which we use results from previous tasks <sup>vengono utilizzati come input per quelle successive</sup> to feed input to subsequent ones
- The ability to orchestrate tasks <sup>è strettamente legata alla</sup> is tightly coupled to the readability and maintainability of code
  - as the layers of asynchronous processes increase in both number and complexity, being able to compose and read code becomes increasingly difficult
  - <sup>ricorda</sup> recall the “callback hell” with the callback model

# 1/2 ASSEMBLY LINE ANALOGY

- We can think of data processed by a reactive application as moving through an ~~assembly line~~ catena di montaggio
    - the ~~raw material~~ materia prima proviene pours from a *source* (the original *Publisher*) and ends up as a finished product ready to be pushed to the *consumer* (or *Subscriber*)
    - the raw material can go through various transformations and other intermediary steps or be part of a larger assembly line that aggregates intermediate pieces together
    - If there is a ~~glitch or clogging~~ malfunzionamento o intasamento at one point (perhaps boxing the products takes a disproportionately long time), the afflicted workstation can signal ~~upstream~~ all'inizio della catena to limit the flow of raw material
- La stazione sovraccarica invia un segnale all'indietro (upstream) verso le stazioni precedenti e la sorgente per dire: "Rallenta! Inviarmi al massimo 5 pezzi alla volta fino a nuovo ordine".



# 2 OPERATORS

Significa che quando concateni un operatore, la libreria reattiva crea un nuovo Publisher che incapsula (avvolge) quello precedente:  
L'operatore B avvolge l'operatore A. Per A, l'operatore B è il suo Consumer (perché B chiede i dati ad A). Per il Subscriber finale, l'operatore B è il suo Producer (perché i dati arrivano da B).

- Operators are like *workstations* in an assembly analogy
  - each operator adds behavior to a Publisher and wraps the previous step's Publisher into a new instance
- The whole chain is ~~thus~~ <sup>quindi</sup> linked, ~~such that data originates from~~ <sup>in modo tale che i dati provengano dal</sup> the first Publisher and moves down the chain, transformed by each link
  - ~~eventually~~ <sup>alla fine</sup>, a Subscriber finishes the process
- Many kind of operators
  - from simple transformation and filtering to complex orchestration and error handling

senza un Subscriber (il consumatore finale), l'intero processo non parte nemmeno e non porta a nessun risultato concreto.

# NOTHING HAPPENS UNTIL SUBSCRIBING

- In a Publisher chain typically data does *not* start pumping into it by default
  - first, the abstract description of the asynchronous process is created (*configuration stage*)
  - then, ~~by subscribing~~<sup>tramite iscrizione</sup>, a Publisher is ~~bound to a Subscriber~~<sup>viene associato ad un</sup>, which *triggers the flow of data in the whole chain*
    - this is achieved internally by a single request signal from the Subscriber that is propagated upstream, all the way back to the source Publisher

# 4 BACKPRESSURE

- Can be described in the assembly line analogy as a feedback signal <sup>inviato al livello superiore</sup> ~~sent up the line~~ when a workstation processes more slowly than an upstream workstation
  - a subscriber can work in <sup>modalità illimitata</sup> ~~unbounded mode~~ and let the source push all the data at its fastest achievable rate
  - or, *it can use the request mechanism to signal the source that it is ready to process at most  $n$  elements* ( $\Rightarrow$  **backpressure**)  
Il consumatore invia un segnale di feedback verso l'alto (upstream) dicendo: "Sono pronto a ricevere al massimo  $n$  elementi". La sorgente non spinge più dati finché il consumatore non elabora quei  $n$  elementi e non invia un nuovo segnale `request(n)`.
- This transforms the push model into a **push-pull hybrid**
  - the downstream can pull  $n$  elements from upstream if they <sup>immediatamente</sup> ~~are readily~~ available
  - but if the elements are not ready, they get pushed by the upstream whenever they are produced
- More later (about ReactiveX)

# HOT VS COLD

- The Rx family of reactive libraries distinguishes two <sup>grandi</sup> ~~broad~~ categories of reactive streams: **cold** and **hot**

- This distinction mainly has to do with how the reactive stream reacts to subscribers:

→ Un flusso è cold quando non fa nulla finché qualcuno non si iscrive (subscribe). Ogni volta che un nuovo utente si iscrive, il flusso ricomincia da capo apposta per lui.

- a *cold* stream starts a new stream for each Subscriber, including at the source of data
  - every subscriber typically gets all the elements of the streams ~~independently from the moment they subscribe~~ <sup>indipendentemente dal momento in cui si è iscritto al flusso</sup>
- a *hot* stream does not start ~~from scratch~~ <sup>da zero</sup> for each Subscriber
  - rather, late subscribers receive signals emitted *after* they subscribed
  - some hot reactive streams can cache or replay the history of emissions totally or partially. <sup>alcuni flussi Hot possono fare "cache or replay": Questo serve a mitigare il problema dei "ritardatari": in replay, un flusso Hot può tenere in memoria gli ultimi N elementi. Quando un nuovo Subscriber arriva, riceve subito quei messaggi passati per "mettersi in pari" e poi prosegue in tempo reale.</sup>
  - from a general perspective, a hot sequence can even emit when no subscriber is listening
    - an exception to “nothing happens before you subscribe”

# AN EXAMPLE: REACTIVE EXTENSIONS

- Rx has been originally introduced as a library on .NET
  - inspired to research work on functional reactive programming (FRP)
- Nowadays "reactive extension" are being developed for almost every programming language
  - <http://reactivex.io/>
    - “*The Observer pattern done right*”
    - “combination of the best ideas from the Observer pattern, the Iterator pattern, and functional programming”
  - Common ref for several lang: RxJS, RxJava, RxScala, RxPython..
  - Adopted by several web/mobile framework
    - both frontend and backend (e.g. Angular)
- Main vision explained in the paper:  
*Erik Meijer. 2012. Your mouse is a database. Commun. ACM 55, 5 (May 2012), 66-73.*

# CORE PROPERTIES

- Senza RP: Se devi fare una chiamata API, aspettare il risultato, e poi usarlo per farne un'altra, devi incastrare diverse Promise o Callback.  
- Con RP: Puoi "concatenare" le operazioni come se fossero anelli di una catena. Puoi dire: "Prendi i dati A, trasformati con la funzione B, e solo dopo uniscili ai dati C". Il codice diventa lineare e leggibile, anche se sotto accadono cose asincrone molto complesse.



- Three core properties:
  - **Asynchronous and event-based**
    - Rx's mission statement is to simplify those programming models.
    - a key aspect for developing reactive user interfaces, reactive web app, cloud app - where asynchrony is quintessential.
  - **Composition**
    - simplifying in particular the composition of asynchronous computations
  - **Observable collections**
    - leveraging the active knowledge of programming model like LINQ, by looking at asynchronous computations as observable data sources
    - A mouse *"becomes a database of mouse moves and clicks"*: such asynchronous data sources are composed using various combinators in the LINQ sense, allowing things like filters, projections, joins, time-based operations, etc.

Usi operatori matematici e logici su eventi che non sono ancora accaduti, trattandoli come se fossero record di una tabella.

# RX PRINCIPLES

- Using Rx, we can
  1. represent multiple *asynchronous data streams* (**observable** streams) coming from diverse sources
    - e.g., stock quote, tweets, computer events, web service requests, etc.
  2. *subscribe* to the event stream using the **IObserver<T>** interface (on .NET)
    - the IObservable<T> interface notifies the subscribed IObserver<T> interface whenever an event occurs.
- Because observable sequences are data streams, we can query them using standard LINQ query operators implemented by the Observable extension methods.
  - thus you can filter, project, aggregate, compose and perform time-based operations on multiple events easily by using these standard LINQ operators.
- In addition, there are a number of other reactive stream specific operators that allow powerful queries to be written.
  - cancellation, exceptions, and synchronization are also handled gracefully by using the extension methods provided by Rx
- Rx complements and interoperates smoothly with both synchronous data streams (IEnumerable<T>) and single-value asynchronous computations (Task<T>) as the following diagram shows

# IOBSERVABLE IN .NET

- Rx complements and interoperates smoothly with both synchronous data streams (`IEnumerable<T>`) and single-value asynchronous computations (`Task<T>`) as the following diagram shows:

|                                     | Single return value        | Multiple return values                   |
|-------------------------------------|----------------------------|------------------------------------------|
| <b>Pull/Synchronous/Interactive</b> | T                          | <code>IEnumerable&lt;T&gt;</code>        |
| <b>Push/Asynchronous/Reactive</b>   | <code>Task&lt;T&gt;</code> | <b><code>IObservable&lt;T&gt;</code></b> |



# OBSERVER/OBSERVABLE INTERFACE

```
interface IObservable<T> {  
    IDisposable Subscribe(IObserver<T> observer);  
}
```

```
interface IObserver<T> {  
    void OnNext(T value);  
    void OnError(Exception error);  
    void OnCompleted();  
}
```

```
interface IDisposable {  
    void Dispose();  
}
```

# SIMPLE EXAMPLES IN .NET

```
...
IObservable<int> source = Observable.Range(1, 10);
IDisposable subscription = source.Subscribe(
    x => Console.WriteLine("OnNext: {0}", x),
    ex => Console.WriteLine("OnError: {0}", ex.Message),
    () => Console.WriteLine("OnCompleted"));
Console.WriteLine("Press ENTER to unsubscribe...");
Console.ReadLine();
subscription.Dispose();
...
```

- When an observer subscribes to an observable sequence, the thread calling the Subscribe method can be different from the thread in which the sequence runs till completion.
- Therefore, the Subscribe call is *asynchronous* in that the caller *is not blocked until the observation of the sequence completes*.

# QUERYING OBSERVABLE SEQUENCES USING LINQ OPERATORS

- Combining streams

```
var source1 = Observable.Range(1, 3);  
var source2 = Observable.Range(1, 3);  
source1.Concat(source2)  
    .Subscribe(Console.WriteLine);
```

the resultant sequence is 1,2,3,1,2,3. This is because when you use the Concat operator, the 2nd sequence (source2) will not be active until after the 1st sequence (source1) has finished pushing all its values. It is only after source1 has completed, then source2 will start to push values to the resultant sequence. The subscriber will then get all the values from the resultant sequence.

- Merge streams

```
var source1 = Observable.Range(1, 3);  
var source2 = Observable.Range(1, 3);  
source1.Merge(source2)  
    .Subscribe(Console.WriteLine);
```

in this case the resultant sequence will get 1,1,2,2,3,3. This is because the two sequences are active at the same time and values are pushed out as they occur in the sources. The resultant sequence only completes when the last source sequence has finished pushing values.

- Projection

```
var seqNum = Observable.Range(1, 5);  
var seqString =  
    from n in seqNum  
    select new string('*', (int)n);  
seqString.Subscribe(  
    str => { Console.WriteLine(str); });  
Console.ReadKey();
```

mapping a stream of num into a stream of strings

arunda

# QUERYING OBSERVABLE SEQUENCES USING LINQ OPERATORS (.NET)

- Other important and useful operators
  - filtering, time oriented operations, handling exceptions...
- In general:
  - **transform** data
    - using methods like aggregation and projection.
  - **compose** data
    - using methods like zip and selectMany.
  - **query** data
    - using methods like where and any.

# REFERENCES

- LEE, E. A. AND MESSERSCHMITT, D. G. 1987. Synchronous Data Flow. In Proceedings of the IEEE. Vol. 75. 1235–1245.
- ELLIOTT, C. AND HUDAK, P. 1997. Functional reactive animation. In Proceedings of the second ACM SIGPLAN International conference on Functional programming. ICFP '97. ACM, New York, NY, USA, 263– 273.
- G. H. COOPER, S. KRISHNAMURTHI, S. 2006. Embedding dynamic dataflow in a call-by-value language. In ESOP'06: Proceedings of the 15th European conference on Programming Languages and Systems. Springer-Verlag, Berlin, Heidelberg, 294–308.
- EDWARDS, J. 2009. Coherent reaction. In OOPSLA '09: Proceeding of the 24th ACM SIGPLAN conference companion on Object oriented programming systems languages and applications. ACM, New York, NY, USA, 925–932.
- Erik Meijer. 2012. Your mouse is a database. Commun. ACM 55, 5 (May 2012), 66-73.
- E. Bainomugisha et al. A Survey on Reactive Programming. ACM Computing Surveys (CSUR). Volume 45 Issue 4, August 2013
- G. Salvaneschi et al. Towards Distributed Reactive Programming. OOPSLA 2014
- R. Mogk et al. Fault-tolerant Distributed Reactive Programming. ECOOP 2018: 1:1-1:26